

Results, with the charts behind them

what was measured, in which unit, and what the unit hides

Abstract

Every number here comes from a run of the shipped code in exact rational arithmetic, cross-checked against an independent reference, and every chart is generated from those numbers by `docs/figs/regenerate.py`. The losses are reported at the same length as the wins, and so is the one finding that outlasts all of them: on this machine wall-clock time cannot resolve the effects being measured, so most of what follows is counted in *work* — sub-problems, branch-and-bound nodes, simplex pivots — and not in seconds.

Contents

1	What is being compared	1
2	The margin, and how it moves with p	2
3	Is the margin real, or is the baseline handicapped?	2
4	Four improvements to the exact search	4
4.1	Why four different units	4
4.2	The measurement lesson	4
5	Hybridisation: what paid and what did not	5
5.1	Seeding the exact search	5
5.2	But as a generator it is the stronger of the two	5
6	The asymmetry, which is the most transferable result	6
6.1	Two ideas that feed the same mechanism do not add	6
7	The complete efficient set, for under one doubling	6
8	Every verdict, including the losses	8
9	How all of this is checked	8

1 What is being compared

Two exact methods answer (P_E) and agree on every instance. They differ in where they keep the unexplored part of the problem, and that decides the cost:

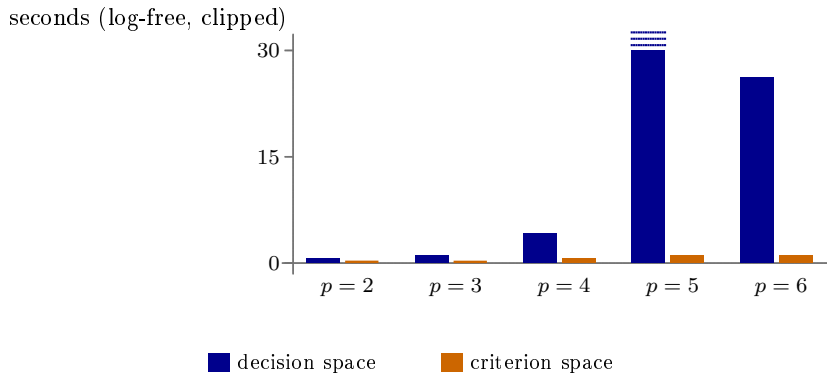
decision space truncates D with Sylva–Crema cuts. Each cut is a disjunction, so it costs p binaries and $p + 1$ big- M rows, and every sub-problem is strictly larger than the last.

criterion space keeps a list of boxes in $\mathbb{R}^p$. The same truncation is a box subtraction; every sub-problem is D plus a few ordinary rows, and **the model never grows**.

2 The margin, and how it moves with p

Table 1: Decision space against criterion space, $n = 6$, four instances per row. All twenty agreed and were proved optimal on both sides.

p	decision space	criterion space	ratio
2	0.61 s	0.35 s	1.74×
3	1.02 s	0.34 s	2.99×
4	4.21 s	0.61 s	6.92×
5	112.77 s	1.09 s	103×
6	26.15 s	1.08 s	24.3×



The bars are clipped at 30s so the small p stay visible; the $p = 5$ decision-space bar runs off the top at 112.77 s.

Two forces grow with p and pull against each other. The split produces p children, so the list branches wider; but the non-dominated set grows too, so the decision-space method needs more cuts, and each cut costs it p binaries and $p + 1$ rows. The measurement settles which dominates, and it settles it against the prediction made before running it: branching multiplies a *count* of cheap, fixed-size models, while the binaries multiply the *cost of one model*, and they accumulate.

On the magnitudes. The direction is solid — the per- p median ratios are 1.35, 3.2, 2.1, 12.6, 15.8, and every instance agreed and was proved optimal. The totals are not stable estimates: four instances per p is few, and 110.95 of the 112.77 seconds at $p = 5$ come from a single instance. The table reports what was measured, not an expected value.

3 Is the margin real, or is the baseline handicapped?

A 103×

 margin — and a 204× one on front enumeration — invites the suspicion that the loser was crippled rather than beaten, so the losing method was profiled instead of merely quoted.

Where the time goes. On the instance that takes 137 s to produce a 30-vector front in decision space:

Over those 31 calls, branch & bound explores 14983 nodes, and the model grows from 5 columns and 7 rows to 95 and 127 — exactly p binaries and $p + 1$ rows per cut. Nothing outside the sub-problems is worth optimising, and the growth of the sub-problems is precisely what the criterion-space method does not have. **The cost is structural.**

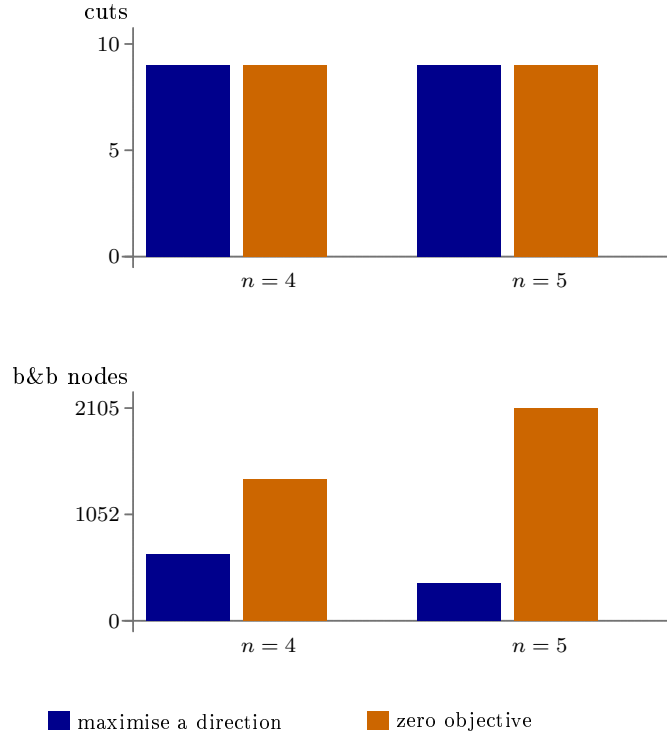
Table 2: Profile of the decision-space enumeration, 137.38 s in total.

	time	share	calls
probe (the integer program)	137.21 s	100%	31
repair to an efficient point	0.09 s	0%	30
building and adding the cut	0.02 s	0%	30

The one candidate handicap, tested. The probe only needs *some* feasible point of the truncated region, yet it maximises a linear direction. Dropping the objective looks strictly cheaper — stop at the first integer point found. Measured, it is four to thirteen times *worse*:

Table 3: Probing by maximising a direction against probing with a zero objective. Same instances, same cuts.

	cuts	b&b nodes	time
$n = 4$, maximise a direction	9	658	1.35 s
$n = 4$, zero objective	9	1397	5.33 s
$n = 5$, maximise a direction	9	372	0.79 s
$n = 5$, zero objective	9	2105	10.15 s



The two charts are the whole finding: the top one is flat and the bottom one is not. **The cut counts are identical** — the direction changes neither how many vectors there are nor the order they come out in — while the branch & bound inside each probe grows by a factor of two to six. A zero objective gives it nothing to prune with: every node’s relaxation is worth 0, so no bound can discard a subtree until an integer point has been stumbled upon.

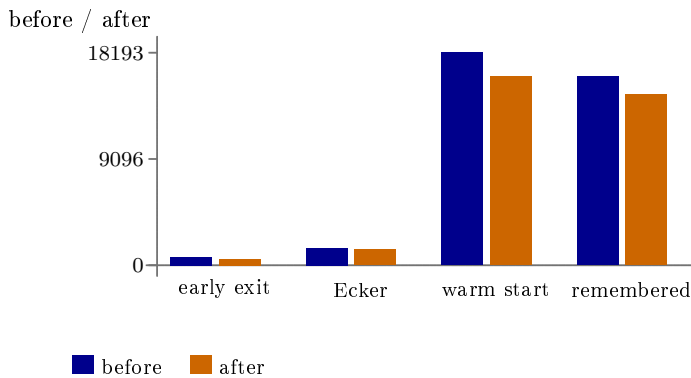
So the direction is load-bearing rather than decorative, the criterion-space method uses the same one, and the margins stand. This is reported at length because it is a plausible-looking optimisation that makes things much worse, and because it was checked while *suspecting the comparison was unfair* — which is the only time such a check is worth anything.

4 Four improvements to the exact search

Each of the four removes work without changing the answer. They are listed with the unit each was measured in, because no single unit sees all four.

Table 4: What each change removes, and what it leaves alone.

change	unit	before → after	
early exit on a hopeless bound	sub-problems	733 → 549	−25%
not re-deriving Ecker & Kouada	b&b nodes	8105 → 7618	−6.0%
warm-starting a box from its parent	simplex pivots	18193 → 16169	−11.1%
cutting on a remembered efficient point	simplex pivots	16169 → 14602	−9.7%



4.1 Why four different units

Because each change removes a different *kind* of work, and a unit that sees one is blind to the others.

- The early exit removes whole sub-problems, so it shows in a box count — but those sub-problems are the *cheapest* ones, each killed by the root relaxation the moment it sees the cutoff, so it barely moves nodes or pivots.
- Ecker & Kouada removes an integer program per inefficient box, so it shows in programs and nodes — but not in the box count, which is identical.
- The warm start removes a simplex phase I per box, so it shows only in pivots: programs, nodes and boxes are all identical.
- The remembered-point shortcut removes an efficiency test *and* changes which boxes are produced, so it is the only one of the four that is not answer-identical by construction and had to be checked against the answer instead.

4.2 The measurement lesson

The wall-clock A/B for the Ecker & Kouada change reads $0.96\times$ — *slower* — and is faster on 9 of 18 instances. That cannot be causal: removing 125 integer programs cannot cost time.

The conclusion is about the instrument. This machine’s noise floor is roughly $\pm 30\%$ per instance *even at per-instance minima over five repeats*, so it cannot resolve a 6% effect, and no number of repeats fixes that. Counting integer programs, nodes and pivots gives the same figure on every run and every machine.

Two habits follow, both adopted only after failing at them first:

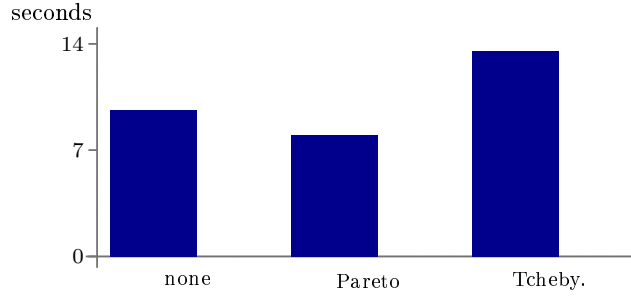
1. A measurement taken while anything else was running on the machine is **discarded, not quoted**. The first A/B for that change was contaminated exactly so and was re-run.
2. A gain predicted from the **mean** cost of a call is not a prediction. The profile showed the efficiency test at 18.4ms per call, the dearest operation in the search — but what the shortcut removes is the repair’s test at 7.68ms, the cheap kind. Reasoning from the wrong row of the right profile overstated the gain by half.

5 Hybridisation: what paid and what did not

5.1 Seeding the exact search

Table 5: Seeding the box search, 18 instances. “Exactly optimal” counts the instances where the seed already attains Φ^* .

seed	total	seed cost	exactly optimal
none	9.64 s	—	—
Pareto local search	7.97 s (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18



The Tchebychev seed is *slower than not seeding at all*, and both halves of that are structural. Its cost is $p + 1$ integer programs on an enlarged model — not the reference point’s fault, since computing z^* took 0.01–0.13s of the seed against 0.13–0.85s for the scalarisations. Its quality is the real finding: the program steers by criterion-space geometry relative to z^* and **never looks at Φ** , while the local search is guided by Φ at every move.

5.2 But as a generator it is the stronger of the two

A weighted sum maximises only at *supported* efficient points. Varying the Tchebychev weights reaches unsupported ones too, which is what the literature uses it for.

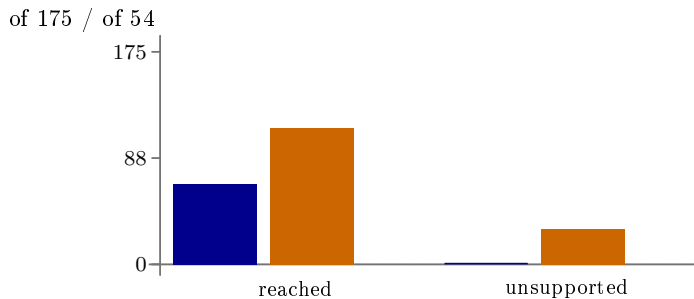


Table 6: Reach from the same grid of 27 weight vectors, 20 random instances.

	efficient points reached	of which unsupported
weighted sum	66/175	0/54
augmented Tchebychev	112/175	29/54

All 540 optima returned were efficient. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, and 1080 optima with none inefficient.

6 The asymmetry, which is the most transferable result

Table 7: The *same* free optimum, handed to each exact method at iteration 1.

exact method	effect	mechanism
criterion-space box search	1.42×, up to 52% fewer boxes	boxes discarded whole
decision-space cut method	zero iterations saved	none

The heuristic is identical in both rows. What differs is whether the exact method has a mechanism through which a lower bound can remove work: a box is compared against the incumbent *before* it is split, and failing that comparison deletes it with every descendant it would have had; what the decision-space method cuts is decided by the efficiency test on its own maximiser, and Φ_{opt} enters only as a cutoff inside a branch and bound that would have found the same point anyway.

So “hybridise an exact method with a metaheuristic” is not one question with one answer, and the mechanism is cheaper to check than the hybrid is to build and measure.

6.1 Two ideas that feed the same mechanism do not add

Table 8: Sub-problems solved over the 18 instances.

	boxes solved	reduction
neither	733	—
early exit only	549	25%
metaheuristic seed only	562	23%
both	539	26%

Either alone removes about a quarter; together they remove about a quarter. On top of the seed the exit is worth 4% rather than 25%, because a good incumbent arrives early enough that the hopeless tail barely forms. Two ideas add only when they feed different mechanisms; two that feed the same one are one idea, measured twice.

7 The complete efficient set, for under one doubling

The front enumeration proves its *vector* list complete and says, in the same breath, that the *point* list is not: one point is kept per slice. Closing that gap turns out to be nearly free, for one reason:

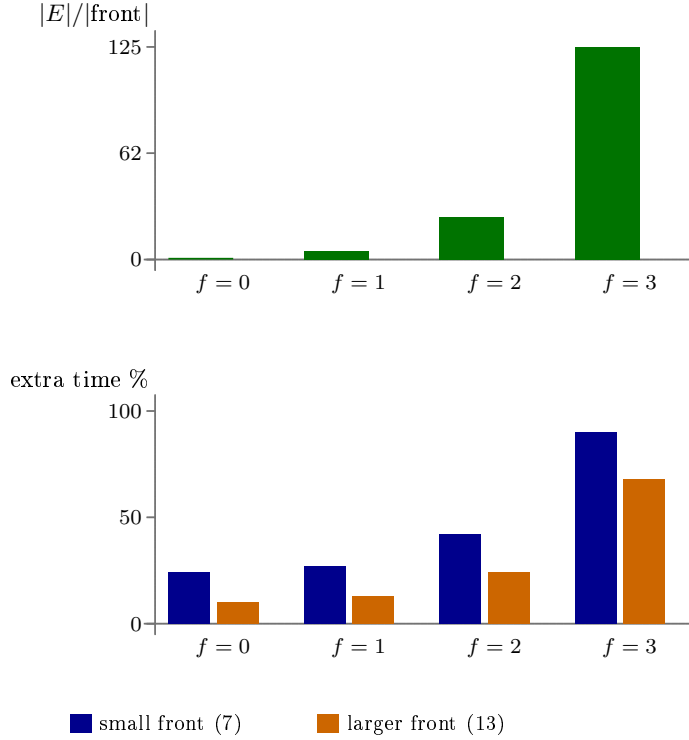
If v is non-dominated, **every** $x \in D$ with $Z(x) = v$ is efficient — a y dominating such an x would give $Z(y) \geq v$ with a strict inequality, making v dominated.

So $E(P_D)$ is exactly the union of the front’s slices, and **no efficiency test is paid on any slice point**: the dearest operation in the package is not invoked at all. A slice is D plus p linear equalities — linear even for fractional criteria, since $d^\top x + b > 0$ clears the ratio — so there is no binary, no big- M , and no growth from one slice to the next.

Below, five variables enter Z and f further ones enter only their own bound $0 \leq x_j \leq 4$: invisible to Z , so each slice is a grid of 5^f points that the front collapses to one.

Table 9: What the slices recover, and what they cost.

f	front	$ E(P_D) $	ratio	slices	extra time
0	7	7	$1\times$	0.016 s	24%
0	13	13	$1\times$	0.018 s	10%
1	7	35	$5\times$	0.019 s	27%
1	13	65	$5\times$	0.024 s	13%
2	7	175	$25\times$	0.036 s	42%
2	13	325	$25\times$	0.050 s	24%
3	7	875	$125\times$	0.103 s	90%
3	13	1625	$125\times$	0.167 s	68%



The two charts are the finding: what is recovered multiplies by five at each step, and the extra time does not. At $f = 3$ the front is *complete and missing 99.2% of the points*, and recovering all 1625 of them never doubles the cost. The scan is output-sensitive — about 0.1 ms per point, against 7.68 ms for a single efficiency test, of which none are paid.

The honest reading, which is conditional. At $f = 0$ the front *already was* the efficient set, and the 10–24% buys only the proof of that — the usual case on generic instances, where 476 efficient points sat on 476 distinct vectors over 32 of them. This is not a speed-up and it is not uniform. What it buys is that the method now *knows* which case it is in, rather than returning a point list whose completeness rested on an assumption about the coefficients.

8 Every verdict, including the losses

Table 10: The fourteen things tried, with what each was worth.

attempt	verdict	effect
decision space $\rightarrow$ criterion space	win	$5.05\times$, up to $173\times$
exact-exact handover ($s = 1$)	tie	8.47 s vs 8.11 s
exact-exact handover ($s = 3$)	loss	13.14 s vs 8.47 s
exact-metaheuristic (Pareto archive)	win	$1.25\times$ of a $1.42\times$ ceiling
exact-Tchebychev seed	loss	$0.71\times$
Tchebychev as a generator	win	reaches unsupported points
incumbent seeding in decision space	loss	zero iterations saved
LP pre-filter on each box	loss	$7.95 \rightarrow 8.36$ s, box counts identical
two-at-a-time split	loss	loses across $p = 3..7$, worse as p grows
early exit on a hopeless bound	partial	25% fewer boxes, no measurable time
early exit <i>on top of</i> the seed	loss	4% rather than 25%: a substitute
warm start + remembered point	win	20% fewer pivots together
zero-objective probe	loss	$4\text{--}13\times$ slower, cuts identical
slices instead of an efficiency test	win	all of $E(P_D)$, under $2\times$

Six are losses and one is a tie. They are reported at the same length as the wins because they carry the same information: a change pays only where the method has a mechanism it can feed, that mechanism is cheaper to identify than the change is to build and measure, and two ideas feeding the same mechanism are one idea counted twice.

9 How all of this is checked

Exact rational arithmetic throughout, no floating point anywhere, no third-party dependency, and 80 tests run on every commit. Several check *properties* rather than answers, since a method that silently lost efficient points would still agree with the optimum on favourable instances: the split is a partition; the metaheuristic never hands over an inefficient point; every remembered point is efficient; a warm root reaches the same value as the cold one it replaces; the early exit leaves boxes open and still proves optimality.

Answers are cross-checked three ways: against the decision-space method, against exhaustive enumeration on small instances, and against a Φ -ordered scan on instances up to 34635 feasible points.